

PROYECTO FIN DE CICLO

DAM/DAW/ASIR

ReserVia

Plataforma Digital de Reservas de Restaurantes con IA

Fase: [Número: 02]

[Francisco Javier Bailon Garcia y Alejandro Chacón Ortega]

Memoria y Documentación Técnica General del Sistema

Los documentos, elementos gráficos, vídeos, transparencias y otros recursos didácticos incluidos en este contenido pueden contener imprecisiones técnicas o errores tipográficos. Periódicamente se realizan cambios en el contenido. Fomento Ocupacional FOC SL puede realizar en cualquier momento, sin previo aviso, mejoras y/o cambios en el contenido. Es responsabilidad del usuario el cumplimiento de todas las leyes de derechos de autor aplicables. Ningún elemento de este contenido (documentos, elementos gráficos, vídeos, transparencias y otros recursos didácticos asociados), ni parte de este contenido puede ser reproducida, almacenada o introducida en un sistema de recuperación, ni transmitida de ninguna forma ni por ningún medio (ya sea electrónico, mecánico, por fotocopia, grabación o de otra manera), ni con ningún propósito, sin la previa autorización por escrito de Fomento Ocupacional FOC SL. Este contenido está protegido por la ley de propiedad intelectual e industrial. Pertenecen a Fomento Ocupacional FOC SL los derechos de autor y los demás derechos de propiedad intelectual e industrial sobre este contenido. Sin perjuicio de los casos en que la ley aplicable prohíbe la exclusión de la responsabilidad por daños, Fomento Ocupacional FOC SL no se responsabiliza en ningún caso de daños indirectos, sean cuales fueren su naturaleza u origen, que se deriven o de otro modo estén relacionados con el uso de este contenido.

© 2023 Fomento Ocupacional FOC SL todos los derechos reservados.

Índice

1. Estudio inicial previo a la realización del proyecto	4
1.1. Clasificar las empresas del sector por sus características organizativas	4
1.2. Poner un ejemplo de estructura organizativa para una empresa del sector	4
1.3. Identificar las necesidades/demandas que cubre el proyecto y asociarlas al cliente	4
1.4. Descripción del proyecto	4
1.5. Justificar el tipo de proyecto elegido para dar solución al problema	4
1.6. Características principales del proyecto elegido	4
2. Identificación de necesidades y diseño del proyecto	5
2.1. Estudio inicial y planificación del proyecto	5
2.2. Aspectos fiscales y laborales	5
2.3. Viabilidad económica	5
2.4. Modelo de solución	6
3. Ejecución del proyecto y pruebas	6
3.1. Riesgos de ejecución del proyecto	6
3.2. Documentación de ejecución	7
3.3. Incidencias	13
4. Pruebas y soporte	13
4.1. Crear documento con las pruebas a realizar	13
4.2. Registro de las pruebas realizadas	14
4.3. Evaluar que el proyecto cumple todo lo requerido	18
5. Sostenibilidad y ODS ReserVia	19
5.1. ODS más relevantes para la actividad	19
5.2. Riesgos y oportunidades que representan los ODS	19
5.3. Acciones para atender los retos ambientales y sociales	20
5.4. Uso de software open source en la plataforma	21

1. Estudio inicial previo a la realización del proyecto

1.1. Clasificar las empresas del sector por sus características organizativas y el tipo de producto o servicio que ofrecen

El sector de restauración y tecnologías hosteleras (FoodTech) se clasifica en tres grandes grupos por estructura organizativa y servicio ofrecido: (1) Establecimientos físicos locales tradicionales (negocios familiares con gestión manual o descentralizada de reservas); (2) Cadenas de restauración y franquicias (gestión centralizada, procesos altamente informatizados); y (3) Proveedores de soluciones tecnológicas SaaS (Software as a Service) de gestión de reservas. ReserVia pertenece a esta última categoría, actuando como un intermediario digital interactivo y plataforma SaaS que optimiza la ocupación de mesas físicas de locales mediante tecnología predictiva.

1.2. Poner un ejemplo de estructura organizativa para una empresa del sector

Para explotar una solución SaaS como ReserVia, la organización propuesta se estructura en: (a) Área de Desarrollo y Sistemas (garantiza el uptime de producción, APIs y mantenimiento del chatbot conversacional de IA); (b) Área de Operaciones e Integración de Clientes (onboarding de locales y soporte técnico en planos 2D/3D); y (c) Área Comercial y Legal (gestión de cobros, contratos, y cumplimiento de normativas como el RGPD y la Ley de Servicios Digitales en la contratación electrónica).

1.3. Identificar las necesidades demandas que cubre el proyecto y asociarlas con las necesidades del cliente.

El proyecto cubre necesidades clave del comensal y el hostelero. El comensal necesita transparencia (saber qué mesa ocupará mediante un plano visual interactivo) y asistencia inteligente para elegir local según cercanía y cocina. El hostelero demanda control operacional en tiempo real, reducción del absentismo de clientes (no-shows) mediante notificaciones de correo automáticas y herramientas eficientes para predecir afluencia y compra de suministros para mitigar pérdidas.

1.4. Descripción del proyecto.

ReserVia es una plataforma de reservas de restaurantes con integración de inteligencia artificial. Permite a los usuarios buscar y descubrir restaurantes, explorarlos en un mapa interactivo (MapLibre GL JS), seleccionar asientos específicos y realizar reservas en tiempo real. Adicionalmente, cuenta con un asistente conversacional RAG por inteligencia artificial que facilita la búsqueda de locales y responde preguntas en lenguaje natural.

1.5. Justificar el tipo de proyecto elegido para dar solución al problema.

Se ha optado por un proyecto tecnológico web full-stack, modular y contenerizado (Docker), debido a que resuelve las problemáticas de integración móvil-escritorio sin forzar la descarga de una app nativa. El uso de Django como núcleo REST y React 19 para la SPA garantiza robustez en transacciones seguras de reservas de mesas e interactividad inmediata en la interfaz de usuario.

1.6. Características principales del proyecto elegido.

- Arquitectura desacoplada basada en API REST (Django REST Framework + JWT).
- Búsqueda geográfica en mapa interactivo con geolocalización basada en MapLibre GL.
- Visualización de aforo interactivo por mesa y tramo de servicio (lunch/dinner) en planos 2D/3D.
- Integración RAG con IA mediante el modelo google/gemma-3-4b-it:free (OpenRouter).
- Soporte multilingüe completo (i18n Español/Inglés).

2. Identificación de necesidades y diseño del proyecto

2.1. Estudio inicial y planificación del proyecto

El proyecto se planifica en base a un modelo ágil estructurado en 5 fases de desarrollo consecutivas para garantizar la completitud. Las fases son: (Fase 1) Conectar Google OAuth en la App Web de Producción; (Fase 2) Secreto de Administrador y Dashboard de Administrador con métricas reales; (Fase 3) Secreto de Owner en GitHub y Dashboard de Propietario con aforo e integración real de planos; (Fase 4) Integración de IA en el Dashboard de Restaurante para analítica predictiva; (Fase 5) Sistema de reseñas de punta a punta con ratings detallados. Cada fase sigue el Protocolo de Integración y Despliegue obligatorio establecido.

2.2. Aspectos fiscales y laborales

La explotación comercial de ReserVia se articula como una Sociedad Limitada (S.L.) de base tecnológica. Aplica el impuesto de sociedades correspondiente y cumple con las normativas fiscales del IVA sobre servicios electrónicos. En materia laboral, el equipo mínimo requiere de ingenieros de sistemas/software contratados bajo el convenio del sector de consultoría y tecnologías de la información.

2.3. Viabilidad económica

El modelo de negocio se sustenta en una tarifa de suscripción mensual baja para restaurantes premium (modelo SaaS) o una pequeña comisión fija por reserva efectiva tramitada. Los costes de infraestructura digital en Railway y APIs de IA (OpenRouter gratuito en primera instancia) son sumamente reducidos, lo que garantiza un punto de equilibrio económico a partir de las primeras 50 suscripciones de restaurantes activos.

2.4. Modelo de solución

La arquitectura del sistema está compuesta por un backend robusto basado en Django 4.2 y Django REST Framework, con autenticación segura mediante JWT (SimpleJWT) y cifrado Fernet (AES-128-CBC + HMAC-SHA256) en la base de datos para campos sensibles (UserProfile.phone y StaffCode.email). El frontend se construye con React 19, TypeScript y Tailwind CSS v4. El almacenamiento relacional en producción utiliza PostgreSQL gestionado en Railway.app, mientras que para el desarrollo local se hace uso de una base de datos ligera SQLite3. El aforo dinámico de mesas y reservas se implementa mediante ranuras (AvailabilitySlot) asignadas de forma automática en transacciones atómicas.

3. Ejecución del proyecto y pruebas

3.1. Riesgos de ejecución del proyecto

3.1.1. Identificación de riesgos

Criterio de nivel: Crítico = probabilidad alta + impacto alto · Alto = impacto alto con probabilidad baja/media · Medio = resto de combinaciones.

ID	Riesgo	Área	Prob.	Impacto	Nivel
R01	Vulnerabilidades de seguridad: sin rate limiting en chat API, sin protección brute force, sin blacklist JWT.	Seguridad	Alta	Alto	Crítico
R02	Pérdida o corrupción de datos de reservas en PostgreSQL (Railway).	Infra.	Baja	Alto	Alto
R03	Fallo del servicio de IA (Claude Haiku) por cambios en API Anthropic o límites de uso.	Integración	Media	Medio	Medio
R04	Incumplimiento de cobertura de tests mínima ($\geq 80\%$ backend / $\geq 75\%$ frontend).	Calidad	Media	Medio	Medio
R05	Regresión en FloorPlan/Seat picker por cambios en React 19 o datos del backend.	Técnico	Media	Alto	Alto
R06	Despliegue fallido en Railway durante el rollout escalonado (10% -> 30% -> 100%).	Despliegue	Baja	Alto	Alto
R07	Desfase temporal por subestimación de mapas interactivos e i18n (EN/ES).	Planif.	Media	Medio	Medio
R08	Tokens JWT en localStorage expuestos a ataques XSS.	Seguridad	Alta	Alto	Crítico

3.1.2. Plan de prevención de riesgos

ID	Medida preventiva	Responsable	Plazo
R01	Rate limiting en /api/chat/, protección brute force en login y blacklist de tokens en logout.	Backend Dev	Sprint actual
R02	Backups automáticos diarios en Railway; plan de restore documentado trimestralmente.	DevOps	Antes de prod.
R03	Fallback en chatbot (mensaje de indisponibilidad) y monitorización de límites Anthropic (migrado a OpenRouter).	Backend Dev	Sprint actual
R04	Cobertura de tests como gate en CI/CD; bloquear merge si no se alcanza el umbral.	QA / Dev	CI setup
R05	Tests de integración para reserva -> seat picker; revisar breaking changes de React 19.	Frontend Dev	Continuo
R06	Validar staging antes del rollout; mantener rama de rollback lista y documentada.	DevOps	Pre-deploy
R07	Revisar estimaciones en cada sprint; priorizar funcionalidad core antes de auxiliares.	PM / Dev	Semanal
R08	Migrar tokens a httpOnly cookies para eliminar vector XSS; añadir cabeceras CSP.	Backend Dev	Prioritario

3.2. Documentación de ejecución

3.2.1. Permisos y requisitos legales

Reservia es una plataforma SaaS alojada en la nube (Railway). Al tratar datos personales de usuarios (nombre, email, teléfono, historial de reservas y preferencias), quedan vigentes las siguientes normativas:

Protección de datos — RGPD / LOPD-GDD: Obligaciones: (1) Política de Privacidad accesible; (2) Registro de Actividades (art. 30 RGPD); (3) Canal ARSOPOL para derechos del interesado; (4) Migrar JWT de localStorage a httpOnly cookies (art. 25 y 32 RGPD); (5) Protocolo de notificación de brechas a la AEPD en <72 h (art. 33 RGPD). La designación de DPD no es obligatoria pero se recomienda.

Servicios digitales — LSSI-CE: Obligatorio publicar Aviso Legal con datos del prestador, Política de Cookies (art. 22.2 LSSI-CE) y garantizar el derecho de oposición a comunicaciones comerciales.

Inteligencia Artificial — AI Act (UE 2024/1689): El chatbot se clasifica provisionalmente como sistema de bajo riesgo al no tomar decisiones autónomas significativas. Obligación de transparencia (art. 50): informar al usuario de que interactúa con una IA. Los datos del chat no deben enviarse a terceros sin consentimiento.

3.2.2. Ejecución del proyecto

El backend de Django está estructurado modularmente en torno a la app api, registrando los 10 modelos de datos relacionales e implementando señales (signals) para automatizar el recálculo dinámico de valoraciones medias de restaurantes y el envío asíncrono de correos electrónicos de confirmación mediante un pool de hilos en emails.py. La API expone endpoints REST estructurados con paginación unificada. El frontend de React implementa Lazy Loading en sus 15 rutas mediante Suspense, utilizando un cliente HTTP base que gestiona la renovación transparente de tokens JWT en caso de expiración (error 401) y realiza reintentos exponenciales automáticos.

3.2.3. Manuales finales

3.2.3.1. Manual de Usuario

Acceso a la aplicación: La aplicación es accesible en producción mediante <https://reservia.up.railway.app> o localmente en <http://localhost:5173>.

Registro de cuenta nueva: (1) Haga clic en 'Sign In' (superior derecha); (2) Seleccione la pestaña 'Register'; (3) Rellene: First name, Last name, Email, Phone (opcional) y Password (min. 6 chars); (4) Clic en 'Register'.

Iniciar y cerrar sesión: En la pestaña 'Login', introduzca email y contraseña. Para cerrar sesión, haga clic en su nombre de perfil y seleccione 'Sign out'.

Búsqueda de restaurantes: Puede escribir el nombre o cocina en la barra de búsqueda superior, o hacer clic en los filtros rápidos de tipo de cocina representados por iconos bajo el buscador (■ Italiana, ■ Japonesa, ■ Steakhouse, ■ Fusión, ■ Saludable, ■ Francesa). El sistema mostrará dinámicamente las tarjetas de restaurantes que coincidan.

Proceso de Reserva: Acceda a la página del restaurante. Rellene el formulario indicando la Fecha, Hora y Comensales (1 a 20 personas) y haga clic en 'Reservar'. Alternativamente, puede abrir el chat de IA ■ e indicar sus deseos en lenguaje natural (ej: *'Quiero una mesa para cenar mañana a las 21:00 para 4 personas comida italiana en Madrid'*). La IA detectará los parámetros y le mostrará un botón interactivo de pre-reserva directa. Confirme su reserva y los datos se guardarán en la base de datos con el estado de 'Confirmed'. Recibirá un correo de confirmación de forma inmediata.

Gestión de Reservas: Desde la pestaña 'My Bookings', el usuario puede consultar el estado de todas sus reservas (Confirmed o Cancelled). Para cancelar, pulse el botón 'Cancel' y el aforo quedará liberado automáticamente para otros comensales.

3.2.3.2. Manual de Instalación

Herramienta	Versión	Local	Docker	Railway
Git	Cualquiera	■	■	■
Python	3.11+	■	—	—
Node.js	18+	■	—	—
npm	9+	■	—	—
Docker	Cualquiera	—	■	—

Vía A: Instalación Local (Desarrollo)

1. Clonar el repositorio: `git clone https://github.com/PresiDeWitt/reservia-proyect.git` && `cd reservia-proyect`

2. Configurar variables de entorno backend: Cree `backend/.env` con:

`SECRET_KEY=dev-key-local`

`DEBUG=True`

`ALLOWED_HOSTS=localhost,127.0.0.1`

`CORS_ALLOWED_ORIGINS=http://localhost:5173`

`OPENROUTER_API_KEY=sk-or-your-key-here`

3. Instalar y lanzar Backend:

`cd backend && python -m venv venv && source venv/bin/activate`

`pip install -r requirements.txt && python manage.py migrate`

`python manage.py seed && python manage.py runserver`

4. Instalar y lanzar Frontend: (en otra terminal)

`cd frontend && npm install && npm run dev`

Vía B: Instalación Automatizada con Docker

1. Cree el archivo `.env` en la raíz del proyecto con variables esenciales (`DEBUG=True`, `SECRET_KEY`, etc.).

2. Ejecute el comando: `docker compose up --build`.

El contenedor se compilará en dos fases, aplicará migraciones, ejecutará la semilla e iniciará Gunicorn en el puerto 80.

Vía C: Despliegue en Producción (Railway.app)

1. Conecte su cuenta de Railway con su repositorio GitHub.

2. Cree un servicio de PostgreSQL de Railway en el proyecto. Se inyectará la variable `DATABASE_URL` automáticamente.

3. Configure las variables de entorno en el servicio del backend en Railway: `SECRET_KEY` (clave segura larga), `DEBUG=False`, `ALLOWED_HOSTS=*.railway.app`, `CORS_ALLOWED_ORIGINS` (URL del frontend en HTTPS) y `OPENROUTER_API_KEY`.

4. Al hacer push a la rama main, Railway compilará el Dockerfile, ejecutará las migraciones y levantará la aplicación.

3.2.3.3. Manual de Configuración y Administración

Variables de entorno: Gestionadas mediante el archivo `.env`. Obligatorio establecer `DEBUG=False` en producción y configurar dominios explícitos en `ALLOWED_HOSTS` y `CORS_ALLOWED_ORIGINS` para prevenir fugas y secuestros de peticiones cross-site.

Gestión de Base de Datos: Comandos esenciales de Django ORM:

- `python manage.py migrate`: Aplica las migraciones de base de datos.
- `python manage.py seed`: Comando idempotente que puebla la base de datos de locales, menús, usuarios de prueba y reservas.
- `python manage.py generate_slots --days N`: Comando crítico de administración que genera los `AvailabilitySlots` para todas las mesas activas en el rango de los próximos N días (por defecto 30 días), basándose en las constantes de servicio (lunch: 13:00-15:00, dinner: 20:00-22:30, intervalos de 30 minutos).

Panel de Administración (Django Admin): Accesible en `/admin/`. Requiere crear un superusuario mediante `python manage.py createsuperuser`. Permite la edición en caliente de las reservas, usuarios, reseñas, perfiles y la configuración de las mesas de cada restaurante.

Configuración de Throttling (Rate Limiting): Los límites de peticiones están centralizados en `settings.py` para prevenir abuso:

- `LoginRateThrottle`: limita accesos fallidos en login.
- `RegisterRateThrottle`: limita el registro spam de usuarios.
- `ChatRateThrottle`: limita el uso del chatbot IA para proteger la cuota de la API Key.

Para redefinir los límites en `settings.py`:

```
REST_FRAMEWORK = { 'DEFAULT_THROTTLE_RATES': { 'login': '30/hour', 'register': '20/hour', 'chat': '60/hour' } }
```

Monitoreo y Diagnóstico: Utilizando la CLI del servidor `reservia-ctl` con privilegios `sudo`:

- Ver estado de contenedores: `sudo reservia-ctl compose ps`
- Auditoría de logs en vivo: `sudo reservia-ctl logs reservia-backend-1 --tail 100 -f`
- Reinicio de servicios específicos: `sudo reservia-ctl compose restart backend`

3.3. Incidencias

Incidencia 1: Fallo de red local en llamadas API (Proxy IPv6): En el arranque inicial en local, el frontend mostraba pantallas en blanco y devolvía errores de conexión al llamar a `/api/*`. *Origen:* El archivo de configuración `vite.config.ts` usaba `target: http://localhost:8000`. En sistemas macOS modernos, 'localhost' se resolvía por red primero como dirección IPv6 loopback (`::1`), impactando contra un contenedor Docker inactivo en lugar del backend local en IPv4. *Resolución:* Se modificó el proxy target forzando la dirección IP literal IPv4: `http://127.0.0.1:8000`.

Incidencia 2: Caída general por JWT corrupto en LocalStorage: Los usuarios no logueados sufrían bloqueos al acceder a secciones públicas del sitio. *Origen:* Si en el `LocalStorage` existía una cadena no válida ('undefined' o token dañado), el cliente API lanzaba un error al parsearlo o enviaba tokens rotos que hacían que Django respondiera 401 en llamadas públicas. *Resolución:* Se modificó `AuthContext.tsx` para validar la estructura del token al cargar la app y borrar automáticamente del storage cualquier token corrupto o que recibiera una respuesta 401 en una petición pública.

4. Pruebas y soporte

4.1. Crear documento con las pruebas a realizar

El plan de pruebas del sistema establece la validación exhaustiva de 4 áreas críticas: (a) Pruebas de Red y Conectividad (CORS, HTTPS, resolución por ID y respuesta de proxies); (b) Pruebas de Carga y Rendimiento (comportamiento de rate limit, estabilidad bajo Unicorn workers); (c) Pruebas de Seguridad (auditoría estática Bandit, npm audit, SQLi, inyecciones de chat y aislamiento de IDOR); y (d) Pruebas de Acceso y Lógica de Negocio (registro, inicio de sesión JWT, control de aforo por slot y cancelación).

4.2. Registro de las pruebas realizadas

ID	Nombre del Test	Herramienta	Evidencia / Resultado Esperado	Estado
NET-01	Endpoints Públicos 200	pytest	Peticiones a /api/restaurants/ devuelven HTTP 200 OK y datos correctos.	PASS
NET-02	Bloqueo de Rutas Privadas	pytest	Petición a /api/reservations/my/ sin JWT es rechazada con HTTP 401 Unauthorized.	PASS
NET-04	Validación CORS	cURL	Peticiones desde dominios no listados en CORS_ALLOWED_ORIGINS reciben HTTP 403.	PASS
LOAD-01	Throttle de Login	k6	Realizar más de 30 logins en 1 minuto activa respuesta HTTP 429 Too Many Requests.	PASS
LOAD-02	Throttle de Chat IA	k6	Llamadas masivas al chat son bloqueadas con HTTP 429 para proteger la API Key.	PASS
SEC-01	Auditoría Bandit (SAST)	Bandit CLI	bandit -r api/ devuelve 0 problemas críticos de seguridad.	PASS
SEC-02	Vulnerabilidades npm	npm audit	Auditoría de dependencias frontend devuelve 0 vulnerabilidades.	PASS
SEC-06	Aislamiento IDOR Reservas	Django Test	Intento de cancelar la reserva de otro usuario devuelve HTTP 404.	PASS
ACC-01	Registro y Login JWT	pytest	El registro crea el User y el UserProfile, y devuelve tokens válidos.	PASS
ACC-03	Renovación de Tokens	pytest	Llamar a token/refresh/ con refresh token devuelve un access token nuevo.	PASS
ACC-07	Redirección por Anonimato	Vitest	Acceso a /my-bookings sin sesión activa redirige a la página principal.	PASS

4.3. Evaluar que el proyecto cumple todo lo requerido

La evaluación técnica final certifica que el proyecto cumple con el 100% de la cobertura de aforo, interactividad de planos y control de lógica exigida. La suite de pruebas de integración y unitarias registra un éxito total: **68 de 68 pruebas superadas satisfactoriamente (68/68 OK)**. Las pruebas se dividen en 40 tests en el backend (verificando modelos, serializadores, API REST de auth, disponibilidad, aforo y rate limit de seguridad) y 28 tests en el frontend (verificando el renderizado de componentes bajo temas claro/oscuro, traducción con i18next y flujo de eventos de navegación).

5. Sostenibilidad y ODS ReserVia

Análisis de los Objetivos de Desarrollo Sostenible aplicados a la plataforma de reservas ReserVia.

5.1. ODS más relevantes para la actividad

ReserVia es una plataforma digital de reservas de restaurantes. Tras analizar el modelo de negocio, los grupos de interés (clientes, restauradores, comunidad local) y el impacto real de la actividad, identificamos tres ODS principales:

ODS 12 — Producción y consumo responsables (principal): Es el ODS con encaje más directo. Una plataforma de reservas reduce el desperdicio alimentario porque permite a los restaurantes planificar compras y producción según las reservas confirmadas. En hostelería se desperdicia aproximadamente un 12% de la comida comprada por previsión deficiente; mejorar la previsión de comensales tiene un impacto medioambiental tangible.

ODS 8 — Trabajo decente y crecimiento económico: ReserVia ofrece herramientas digitales (panel de gestión, métricas, reservas online) a pequeñas y medianas empresas de restauración que normalmente no pueden permitirse software propio. Apoya la digitalización del comercio local y la generación de empleo cualificado en el sector hostelero.

ODS 11 — Ciudades y comunidades sostenibles: El mapa explorador y las recomendaciones por ubicación impulsan el turismo gastronómico de proximidad. Promueve los negocios de barrio frente a grandes cadenas y ayuda a descongestionar zonas turísticas saturadas.

Como ODS secundarios mencionables están el ODS 9 (innovación e infraestructura digital) y el ODS 5 (igualdad de género) si se publican políticas internas de inclusión.

5.2. Riesgos y oportunidades que representan los ODS

Cada ODS aporta oportunidades de impacto positivo, pero también riesgos si la plataforma no se gestiona con criterio sostenible. La siguiente tabla resume los principales:

ODS	Riesgos	Oportunidades
ODS 12	Si se permiten cancelaciones de última hora sin coste, el desperdicio alimentario aumenta. Reservas falsas o no-shows generan stock no consumido.	Política de depósitos, recordatorios automáticos 24 h antes, y un distintivo de 'restaurante sostenible' para los locales con prácticas km 0, ecológicas o sin plástico de un solo uso.
ODS 8	Dependencia excesiva de los restaurantes de un único canal digital; riesgo de comisiones abusivas como el modelo de las grandes plataformas de reservas.	Modelo de comisiones justo, formación digital al restaurador, panel con datos abiertos al dueño para que pueda tomar decisiones basadas en evidencia.
ODS 11	Saturar zonas turísticas y desplazar al vecindario; uniformización de la oferta gastronómica.	Promocionar restaurantes de barrios menos visitados, integrar transporte público en las indicaciones de ruta y dar visibilidad a cocinas tradicionales locales.

5.3. Acciones para atender los retos ambientales y sociales

5.3.1. Acciones desde la actividad profesional (ReserVia)

Ambientales:

- Sistema de recordatorio y confirmación 24 h antes para reducir no-shows y, con ello, comida tirada.
- Filtro y badge 'restaurante sostenible' (km 0, ecológico, sin plásticos de un solo uso, opción vegetariana destacada).
- Mostrar opción vegetariana o vegana en cada carta para incentivar el consumo de menor huella de carbono.
- Hosting con energía renovable (proveedores como Hetzner, OVH eco o Cloudflare con compensación).

Sociales y económicas:

- Comisiones reducidas o gratuitas para restaurantes pequeños y familiares por debajo de un umbral mensual de reservas.
- Panel de gestión accesible y traducido a varios idiomas (i18n EN/ES) para reducir la brecha digital.
- Cumplimiento de accesibilidad WCAG AA: lectores de pantalla, contraste suficiente y navegación completa por teclado.

5.3.2. Acciones desde el entorno personal y del equipo

- Trabajo en remoto del equipo de desarrollo: menos desplazamientos, menor huella de transporte.
- Code reviews y formación interna sobre eficiencia de código: un código que consume menos CPU se traduce en menos energía consumida en servidores.
- Reducción consciente del consumo digital innecesario: imágenes optimizadas, lazy loading, evitar dependencias pesadas.
- Compromiso personal del equipo con consumo responsable y formación continua en sostenibilidad aplicada al software.

5.4. Uso de software open source en la plataforma

ReserVia se construye íntegramente sobre tecnologías de código abierto, y mantenemos ese compromiso a futuro. Las principales tecnologías utilizadas son:

Frontend:

- React, Vite y TypeScript (licencia MIT): base de la interfaz de usuario.
- Tailwind CSS (MIT): sistema de estilos.
- React Router, Framer Motion, i18next, React Leaflet (MIT): navegación, animaciones, internacionalización y mapas.

Backend:

- Django y Django REST Framework (BSD): servidor y API.
- SimpleJWT (MIT): autenticación basada en tokens.
- Python (PSF): lenguaje del backend.

Datos y mapas:

- OpenStreetMap: datos cartográficos abiertos y colaborativos en lugar de soluciones propietarias.
- SQLite y PostgreSQL: sistemas de bases de datos libres.